

Welcome Offer Access to everything. Now 30% off. [Upgrade now](#)

Understanding React's `useEffect` Hook: More Than Just Running Code After Render



Devendra Dhote

Follow

3 min read · 2 days ago



66



1



...



Understanding useEffect in React

A complete guide to **side effects**,
dependencies, and **cleanup**.

Common Side Effects



API Calls



Event Listeners



Timers



Local Storage



WebSocket



DOM Updates



```
useEffect(() => {  
  // Side effect (e.g. API call, event listener)  
  fetch('/api/data')  
    .then(res => res.json())  
    .then(setData);  
  
  return () => {  
    // Cleanup (e.g. remove listener, clear timer)  
    // cancel requests, etc.  
  };  
}, [dependencies]);
```

The Lifecycle of useEffect



Remember: `useEffect` is for **synchronizing** your component with **external systems**, not for handling data flow.

When I first started learning React, I used to hear the same sentence everywhere:

"useEffect runs after the component renders."

Although this is true, I always felt it was an incomplete explanation.

The real purpose of `useEffect` is much more interesting.

It isn't just about running code after rendering.

It's about **synchronizing your React component with the outside world.**

Let's understand what that actually means.

. . .

What is `useEffect` ?

`useEffect` is a React Hook that allows us to perform **side effects** after React has updated the UI.

A side effect is simply any operation that happens **outside React's rendering process.**

Some common examples are:

- Fetching data from an API
- Updating `document.title`

- Accessing Local Storage
- Starting timers (`setTimeout` / `setInterval`)
- Adding event listeners
- Opening WebSocket connections

These operations are not part of rendering the UI, which is why React provides the `useEffect` hook.

. . .

Why Do We Need `useEffect` ?

React components should remain **pure**.

Their job is simple:

- Receive props
- Read state
- Return JSX

Imagine writing this inside a component:

```
function App() {  
  fetch("/users");
```

```
  return <h1>Hello</h1>;  
}
```

Looks harmless.

But every time the component re-renders, the API request will run again.

That's obviously not what we want.

Instead, React lets us separate rendering from side effects.

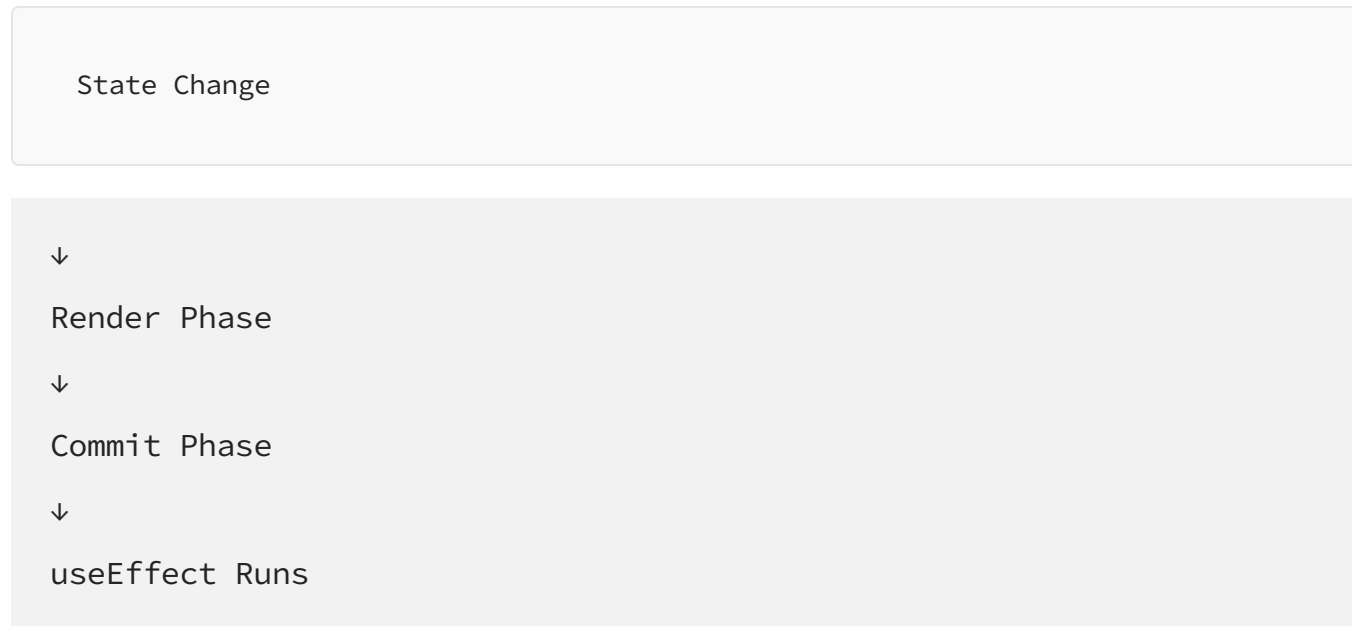
```
useEffect(() => {  
  fetch("/users");  
}, []);
```

Now the API call happens after React finishes rendering the UI.

• • •

The Render Flow

Whenever state changes, React follows this flow:



Notice something important.

`useEffect` never runs during rendering.

It always runs **after** React commits the UI updates to the DOM.

. . .

Different Ways to Use `useEffect`

1. Without a Dependency Array

```
useEffect(() => {  
  console.log("Running...");  
});
```

This effect runs **after every render**.

Flow:

Render

↓

Commit

↓

Effect

. . .

2. With an Empty Dependency Array

```
useEffect(() => {  
  console.log("Runs once");  
}, []);
```

This effect runs only once after the initial render.



This is commonly used for:

- Initial API calls

- Initial setup
- Event listeners

• • •

3. With Dependencies

```
useEffect(() => {  
  console.log(count);  
}, [count]);
```

Now the effect only runs whenever `count` changes.

If `count` remains the same, React skips the effect.

The dependency array tells React:

“Run this effect only when one of these values changes.”

• • •

4. Multiple Dependencies

```
useEffect(() => {  
  console.log("User or Theme changed");  
}, [user, theme]);
```

The effect runs whenever **either** `user` or `theme` changes.

. . .

Cleanup Function

Sometimes an effect creates resources that need to be cleaned up.

For example:

```
useEffect(() => {  
  const id = setInterval(() => {  
    console.log("Running...");  
  }, 1000);
```

```
    return () => {  
      clearInterval(id);  
    };  
  }, []);
```

The returned function is called the **Cleanup Function**.

React executes it:

- Before running the next effect
- When the component unmounts

This helps prevent memory leaks and duplicate subscriptions.

• • •

A Common Misconception

Many developers think:

| "useEffect triggers rendering."

Actually, it's the opposite.

The correct flow is:



Rendering determines whether an effect should run — not the other way around.

• • •

When Should You Use `useEffect` ?

`useEffect` is perfect for tasks like:

- Fetching API data
- Updating the page title
- Working with Local Storage
- Timers
- Event listeners
- Browser APIs
- WebSocket connections

These are all examples of synchronizing React with something outside React.

• • •

When You Should Avoid `useEffect`

One mistake I made while learning React was using `useEffect` for simple calculations.

For example:

```
useEffect(() => {  
  setFullName(firstName + " " + lastName);  
}, [firstName, lastName]);
```

This isn't necessary.

Instead, we can simply write:

```
const fullName = firstName + " " + lastName;
```

Not every piece of logic belongs inside `useEffect`.

If you can calculate something during rendering, you probably don't need an effect.

• • •

Final Thoughts

The biggest lesson I learned about `useEffect` is that it isn't a lifecycle replacement.

It's a **synchronization hook**.

Whenever your React component needs to communicate with something outside React — whether that's an API, the browser, local storage, or a timer — `useEffect` is the tool designed for that job.

Once I stopped thinking of it as “code that runs after render” and started thinking of it as “a way to synchronize React with external systems,” the hook became much easier to understand.

Happy coding champs ❤️ .

JavaScript ✓

React ✓

Programming ✓

Web Development ✓